

Rapport Technique

Banking Application

API RESTful de Gestion Bancaire

Spring Boot 3.2 • PostgreSQL • Render • Neon

Auteur Oberlin Keyanfe Djune
Module INF352, Assurance Qualité et Tests Logiciels
Année 2025–2026
Version 1.0

URL : <https://tp352.onrender.com/swagger-ui.html>

Introduction

Contexte

Le présent rapport technique documente l'architecture, la conception, l'implémentation et le déploiement de l'application **Banking Application**, un système bancaire exposé sous forme d'API RESTful développé dans le cadre du module INF352 — Assurance Qualité et Tests Logiciels.

Objectifs du système

Le système répond aux exigences fonctionnelles suivantes :

- Création et gestion des utilisateurs (nom, email, téléphone).
- Authentification par token JWT.
- Gestion des banques partenaires.
- Création et gestion des comptes bancaires.
- Opérations de dépôt sur compte.
- Retraits d'un compte vers un compte d'un autre utilisateur.
- Transferts entre les comptes d'un même utilisateur.
- Documentation interactive de l'API via Swagger.

Environnement Technologique

Langage et Framework

Composant	Spécification
Langage	Java 17 (LTS)
Framework	Spring Boot 3.2.0
Build	Maven 3.8 avec Maven Wrapper (<code>mvnw</code>)
Serveur d'application	Apache Tomcat 10.1 (intégré)

TABLE 1 – Stack applicative

Persistence des données

Composant	Spécification
ORM	Spring Data JPA / Hibernate 6.3
Base de développement	H2 Database (mode mémoire)
Base de production	PostgreSQL 15 (Neon.tech)
Pool de connexions	HikariCP
Stratégie DDL	<code>create-drop</code> (dev) / <code>update</code> (prod)

TABLE 2 – Infrastructure de données

Bibliothèques principales

Bibliothèque	Version	Rôle
springdoc-openapi	2.3.0	Documentation Swagger
jjwt-api / jjwt-impl / jjwt-jackson	0.12.3	Gestion des tokens JWT
postgresql	–	Driver JDBC PostgreSQL
h2	–	Base de développement
spring-boot-starter-validation	–	Validation des entités
spring-boot-starter-test	–	Framework de test

TABLE 3 – Dépendances Maven

Architecture Logicielle

Architecture en couches

Couche	Composants	Responsabilité
Présentation	Contrôleurs REST	Réception des requêtes HTTP, validation
Métier	Services	Logique applicative, règles de gestion
Accès données	Repositories JPA	Opérations CRUD, requêtes personnalisées
Modèle	Entités JPA	Mapping objet-relationnel

TABLE 4 – Architecture en couches

Diagramme de classes

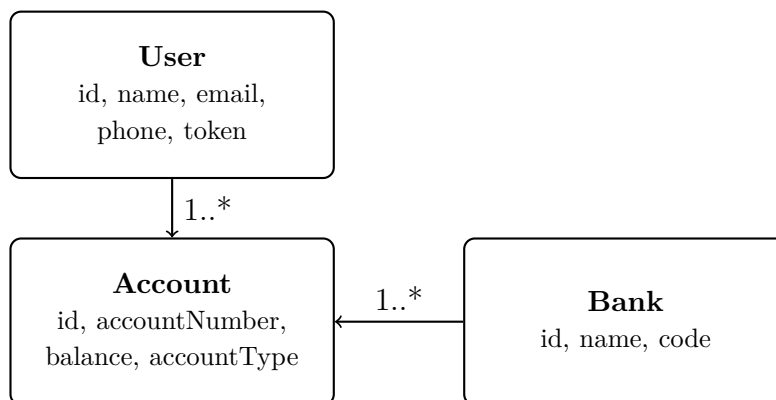


FIGURE 1 – Relations entre les entités principales

Structure des packages

Package	Contenu
com.banking.config	SwaggerConfig, CorsConfig
com.banking.controller	Contrôleurs REST
com.banking.model	User, Bank, Account
com.banking.repository	Repositories JPA
com.banking.service	Services métier
com.banking.exception	Gestion des exceptions

TABLE 5 – Organisation du code source

Modèle de Données

Entité : User

Colonne	Type SQL	Contrainte	Description
id	BIGINT	PRIMARY KEY	Identifiant unique
name	VARCHAR(255)	NOT NULL	Nom complet
email	VARCHAR(255)	UNIQUE, NOT NULL	Adresse email
phone	VARCHAR(255)	UNIQUE	Numéro de téléphone
token	VARCHAR(500)	–	Token JWT actif
created_at	TIMESTAMP	–	Date de création
updated_at	TIMESTAMP	–	Date de modification

TABLE 6 – Structure de la table **users**

Entité : Bank

Colonne	Type SQL	Contrainte	Description
id	BIGINT	PRIMARY KEY	Identifiant unique
name	VARCHAR(255)	UNIQUE, NOT NULL	Nom de la banque
code	VARCHAR(255)	UNIQUE, NOT NULL	Code bancaire
created_at	TIMESTAMP	–	Date de création

TABLE 7 – Structure de la table **banks**

Entité : Account

Colonne	Type SQL	Contrainte	Description
id	BIGINT	PRIMARY KEY	Identifiant unique
account_number	VARCHAR(255)	UNIQUE, NOT NULL	Numéro de compte
balance	NUMERIC(38,2)	NOT NULL, DEFAULT 0	Solde du compte
account_type	VARCHAR(255)	–	CHECKING ou SAVINGS
user_id	BIGINT	FOREIGN KEY	Référence utilisateur
bank_id	BIGINT	FOREIGN KEY	Référence banque
created_at	TIMESTAMP	–	Date de création
updated_at	TIMESTAMP	–	Date de modification

TABLE 8 – Structure de la table **accounts**

Relations entre entités

- Un **User** possède 0 à plusieurs **Account** (relation OneToMany).
- Une **Bank** possède 0 à plusieurs **Account** (relation OneToMany).
- Un **Account** appartient à un seul **User** (relation ManyToOne).
- Un **Account** appartient à une seule **Bank** (relation ManyToOne).

Spécification de l'API REST

Endpoints publics (sans authentification)

Méthode	Endpoint	Description
POST	/api/register	Création d'un compte utilisateur
POST	/api/login	Connexion utilisateur
GET	/api/banks	Liste des banques disponibles
GET	/api/banks/{id}	Détail d'une banque
POST	/api/banks	Création d'une banque

TABLE 9 – Endpoints publics

Endpoints sécurisés (token JWT requis)

Méthode	Endpoint	Description
GET	/api/accounts	Liste des comptes de l'utilisateur
POST	/api/accounts	Création d'un compte bancaire
GET	/api/accounts/{num}	Détail d'un compte
DELETE	/api/accounts/{id}	Suppression d'un compte
POST	/api/accounts/deposit	Dépôt sur un compte
POST	/api/transactions/withdraw	Retrait vers un autre utilisateur
POST	/api/transactions/transfer	Transfert entre ses comptes

TABLE 10 – Endpoints sécurisés

Format des échanges

Tous les échanges se font au format JSON. L'authentification utilise le header HTTP **Authorization** avec la valeur du token JWT. Les dates sont au format ISO 8601. Les montants utilisent le type `BigDecimal` pour une précision financière optimale.

Documentation Swagger

La documentation interactive est accessible à l'adresse `/swagger-ui.html`. Un bouton **Authorize** permet d'injecter automatiquement le token JWT dans toutes les requêtes.

Règles de Gestion et Algorithmes

Gestion des utilisateurs

- RG-1.** L'inscription requiert un nom, un email et un numéro de téléphone.
- RG-2.** L'email doit être unique dans le système.
- RG-3.** Le numéro de téléphone doit être unique dans le système.
- RG-4.** L'identifiant, la date de création et la date de mise à jour sont générés automatiquement.

Gestion des banques

- RG-5.** La création d'une banque requiert un nom et un code.
- RG-6.** Le code bancaire doit être unique dans le système.

Gestion des comptes bancaires

- RG-7.** Un compte est créé avec un solde initial de zéro (0,00).
- RG-8.** Le numéro de compte est généré automatiquement au format ACC-XXXXXXXX.
- RG-9.** Les types de compte disponibles sont CHECKING (courant) et SAVINGS (épargne).

Règles de transaction

- RG-10.** Tout dépôt requiert un montant strictement positif.
- RG-11.** Le dépôt n'est autorisé que sur un compte appartenant à l'utilisateur authentifié.
- RG-12.** Un retrait (vers un autre utilisateur) requiert :
- Un solde suffisant sur le compte source.
 - Des comptes source et destination différents.
 - Un montant strictement positif.
- RG-13.** Un transfert (entre ses propres comptes) requiert :
- L'utilisateur doit être propriétaire des deux comptes.
 - Les comptes source et destination doivent être différents.
 - Le solde du compte source doit être suffisant.
 - Le montant doit être strictement positif.
- RG-14.** Toute transaction nécessite un token JWT valide et non expiré (validité : 24 heures).

Séquence de traitement : Retrait

1. Réception du token, du compte source, du compte destination et du montant.
2. Validation du token JWT et extraction de l'utilisateur.
3. Vérification de l'existence du compte source.
4. Vérification de l'existence du compte destination.
5. Vérification que l'utilisateur est propriétaire du compte source.
6. Vérification que les comptes source et destination sont différents.
7. Vérification que le solde du compte source $\geq$ montant.
8. Vérification que le montant est strictement positif.
9. Débit du compte source : $\text{balance}_{\text{source}} \leftarrow \text{balance}_{\text{source}} - \text{montant}$.
10. Crédit du compte destination : $\text{balance}_{\text{dest}} \leftarrow \text{balance}_{\text{dest}} + \text{montant}$.
11. Sauvegarde atomique des deux comptes (opération transactionnelle).
12. Retour d'une réponse de succès au client.

Sécurité et Authentification

Mécanisme d'authentification

L'application utilise le standard **JWT (JSON Web Token)** :

- Algorithme de signature : HMAC-SHA256 (HS256).
- Clé secrète stockée dans le `TokenService`.
- Durée de validité : 24 heures (86 400 000 ms).
- Payload : `userId`, `email`, `name`, `iat`, `exp`.

Flux d'authentification

1. L'utilisateur s'inscrit (POST `/api/register`) ou se connecte (POST `/api/login`).
2. Le serveur génère un token JWT signé et le stocke.
3. L'utilisateur inclut le token dans le header `Authorization`.
4. Le `TokenService` valide la signature, l'expiration et retrouve l'utilisateur.
5. En cas d'échec, une réponse HTTP 401 (Unauthorized) est retournée.

Gestion des erreurs

Toutes les exceptions sont interceptées par le `GlobalExceptionHandler` :

```
{
  "timestamp": "2026-05-07T12:00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Solde insuffisant"
}
```

Listing 1 – Format standardisé des réponses d'erreur

Déploiement et Infrastructure

Environnement de développement

Paramètre	Valeur
Profil Spring	dev
Base de données	H2 Database (mémoire)
URL de connexion	jdbc:h2:mem:bankingdb
Port du serveur	8080
Console H2	http://localhost:8080/h2-console
Swagger UI	http://localhost:8080/swagger-ui.html
Stratégie DDL	create-drop
Données initiales	data.sql (3 banques prédéfinies)

TABLE 11 – Configuration de développement

Environnement de production

Paramètre	Valeur
Profil Spring	prod
Hébergeur	Render.com (Frankfurt)
URL publique	https://tp352.onrender.com
Base de données	Neon.tech (PostgreSQL 15)
Driver JDBC	org.postgresql.Driver
Pool de connexions	HikariCP (max 5, min 2)
Stratégie DDL	update
Déploiement	Continu (push sur main)

TABLE 12 – Configuration de production

Variables d'environnement

Variable	Description
DATABASE_URL	URL JDBC complète avec <code>?sslmode=require</code>
DATABASE_USERNAME	Nom d'utilisateur Neon
DATABASE_PASSWORD	Mot de passe Neon
PORT	Port d'écoute (8080)

TABLE 13 – Variables d'environnement sur Render

Schéma de déploiement

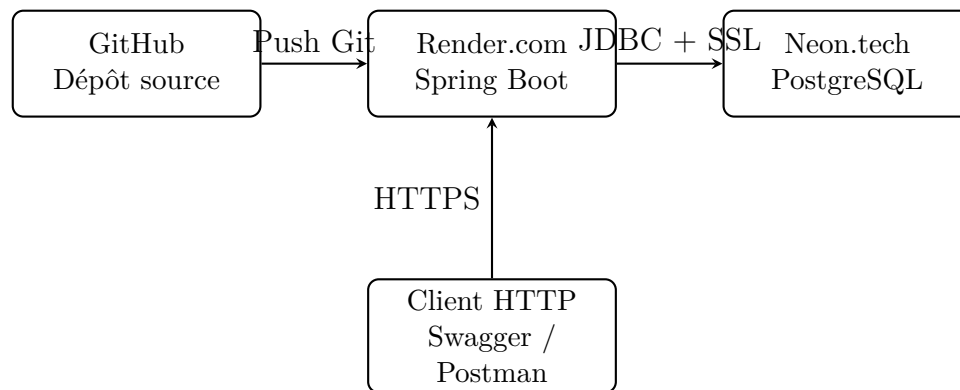


FIGURE 2 – Architecture de déploiement

Procédures Techniques

Lancement en local

1. Cloner le dépôt : `git clone https://github.com/oberlinkeyanfe-ops/TP352.git`
2. Se positionner dans le dossier : `cd TP352`
3. Lancer l'application : `./mvnw spring-boot:run`
4. Accéder à Swagger : `http://localhost:8080/swagger-ui.html`

Déploiement en production

1. Effectuer les modifications en local.
2. Commiter : `git add . && git commit -m "message"`
3. Pousser : `git push origin main`
4. Render détecte le push et redéploie automatiquement.

Inventaire des Fichiers Sources

Fichier	Rôle
BankingApplication.java	Point d'entrée Spring Boot
config/SwaggerConfig.java	Configuration OpenAPI / Swagger
config/CorsConfig.java	Configuration CORS
controller/TokenController.java	Endpoints d'authentification
controller/UserController.java	Endpoints utilisateurs
controller/BankController.java	Endpoints banques
controller/AccountController.java	Endpoints comptes
controller/TransactionController.java	Endpoints transactions
model/User.java	Entité JPA utilisateur
model/Bank.java	Entité JPA banque
model/Account.java	Entité JPA compte bancaire
repository/UserRepository.java	Accès données utilisateurs
repository/BankRepository.java	Accès données banques
repository/AccountRepository.java	Accès données comptes
service/TokenService.java	Service de tokens JWT
service/UserService.java	Service utilisateurs
service/BankService.java	Service banques
service/AccountService.java	Service comptes
service/TransactionService.java	Service transactions
exception/GlobalExceptionHandler.java	Gestion globale des exceptions
exception/ResourceNotFoundException.java	Exception ressource introuvable
application.properties	Configuration Spring (dev)
application-prod.properties	Configuration Spring (prod)
data.sql	Données initiales

TABLE 14 – Inventaire des fichiers sources

Conclusion

Le système **Banking Application** fournit une API RESTful complète pour la gestion bancaire, couvrant l'ensemble du cycle de vie des utilisateurs, des banques, des comptes et des transactions financières.

L'architecture en couches garantit une séparation claire des responsabilités, facilitant la maintenance et l'évolution du code. Le déploiement sur Render avec une base de données Neon assure une disponibilité continue. La documentation Swagger interactive permet une prise en main rapide de l'API.

<https://tp352.onrender.com/swagger-ui.html>